A1 · 学生实操手册 —— Trae SOLO 上手 + 让 Trae 生成并保存 C/S 图书馆 Agent PRD

> **对应**:CS 实训(C/S 模式 Java 图形界面开发)· Vibe Coding 单元① **用法**:跟着一步一步做,每一步都有【操作】【✅ 怎么算做好】【卡住怎么办】。老师带做时你同步做,课后自己做也照这份来。 **前置**:Trae 已安装并登录;本机已装 JDK 11、MySQL 8(`127.0.0.1:3306`,账号 `root`,密码 `12345678`)。

今天要做什么(先读这一段)

这门实训要覆盖 **Swing 图形界面 + JDBC + Socket 网络编程 + 多线程 + 事务 + 局域网 C/S** 一大堆技术点。我们不逐个手写语法去啃,而是用一套真实、能局域网多机演示的 **C/S 图书馆借阅系统**把它们全串起来:**由 AI(Trae SOLO)施工,由你指挥和把关。**

今天不写业务代码,今天写的是"给 AI 的图纸"。 这份图纸叫 **Agent PRD**(给 AI 看的项目说明书),它决定了后面四个单元 AI 生成的全部质量——图纸清楚,AI 生成得准;图纸含糊,AI 只能瞎猜。这就是本课最重要的一句话:**规范是新的源代码。**

一个关键点:这份 PRD 不用你手抄。 Trae 能自己建文件、写内容、存盘,所以你把需求理清楚后,**让 Trae 帮你整理成 PRD 并保存**;你的活是**检查它写得对不对、缺的补上**。这正是 Vibe Coding 的分工:**你负责想清楚 + 说清楚 + 把关,整理和落地交给 AI。**

今天分五步走(外加一个课后任务):

步骤	你要做的事	产出
实操 1	Trae SOLO 上手	能在 SOLO 里对话,AI 有回应且能建/存文件
实操 2	城市探索热身(纯前端上手)	一个能看能点的城市探索小页面(只今天做一次)
实操 3	反向提问梳理 C/S 图书馆需求	需求被一问一答理清楚
实操 4	让 Trae 生成并保存 PRD	`.trae/rules` 文件出现,里面是 Trae 整理的八块 PRD
实操 5	检查 + 微调(审查是你的活)	对照标准审查过、C/S 三条关键规则补齐的 PRD

> 💡 先记住一个 **C/S 心法**:我们这套系统是"客户端-服务器"三段式——**客户端(Swing 界面)不直接连数据库**,而是通过网络(Socket)把请求发给**服务端**,服务端查数据库、处理业务,再把结果发回来。今天写 PRD,就是把这个架构和它的规矩固定下来。

实操 1 · Trae SOLO 上手

Trae 是什么:一个带了 AI 助手的编程 IDE(编辑器)。你用自然语言告诉它要做什么,它自己建文件、写代码、给你看结果,还能在里面编译运行。本课只用它的 **SOLO 模式**——一个模式就够我们做完整个 C/S 图书馆。

核心就一句话:SOLO 模式 = 你说人话,它写代码并保存文件。 你不用管它怎么建文件、怎么配环境,你只管把"要什么"说清楚。

步骤 1.1 · 确认装好并登录

【操作】

- 打开 Trae,确认能看到主界面:一般分三块——**左边文件区 / 中间编辑区 / 右边(或侧边)AI 对话面板**。
- 确认右上角(或设置里)显示已登录状态,不是"请登录 / Sign in"。
- 顺手确认本机环境:命令行敲 `java -version` 能看到 11.x;MySQL 服务已启动(本课用 `127.0.0.1:3306`,root / 12345678)。

【✅ 怎么算做好】

- Trae 正常打开,能看到 AI 对话面板,处于已登录状态;`java -version` 显示 11。

【卡住怎么办】

- **打不开 / 没装**:按预习包《Trae 安装配置指南》重装;或举手找助教。实在装不上,先和邻座**两人一组共用一台**,别卡在装环境上耽误后面的课。
- **`java -version` 不是 11 / 找不到**:装 JDK 11 并配好环境变量;C/S 项目单元②起要编译运行 Java,今天先确认好。
- **MySQL 连不上**:确认 MySQL 服务已启动;今天只是确认,真正建库在单元②。

步骤 1.2 · 新建一个工作区

【操作】

- 在电脑上先建一个**空文件夹**当项目目录(名字随意,比如 `library-cs`)。
- 在 Trae 里选择"打开文件夹 / Open Folder",把这个空文件夹打开成当前工作区。

【✅ 怎么算做好】

- Trae 左边文件区显示的是你刚建的这个文件夹(现在是空的,正常)。

【卡住怎么办】

- **找不到"打开文件夹"**:一般在左上角菜单或欢迎页有"Open Folder / 打开文件夹"入口。
- **不知道文件夹建哪**:建在桌面或一个你找得到的目录即可,路径里尽量别带中文和空格。

步骤 1.3 · 切到 SOLO 模式

【操作】

- 在 AI 对话面板找到模式切换入口,切到 **SOLO** 模式。
- 不同版本入口位置略有差别,一般在对话框上方或旁边有个模式选择的按钮/下拉。

【✅ 怎么算做好】

- 当前对话面板显示的是 SOLO 模式。

【卡住怎么办】

- **找不到 SOLO**:看对话框附近的模式切换按钮/下拉逐个点开看;还找不到就举手,助教会指给你看当前版本的入口位置。

步骤 1.4 · 认识对话框(先试一句)

【操作】

- 在 SOLO 对话框里输入一句话试试水,比如:

  > 你好,请用一句话介绍一下你能帮我做什么。

- 回车,等 AI 回应。

【✅ 怎么算做好(实操 1 的验收点)】

- **你能在 SOLO 输入框里输入内容,并收到 AI 的文字回应。** 到这一步,实操 1 就通过了——说明 Trae SOLO 已经能正常用。

【卡住怎么办】

- **发出去没反应 / 一直转圈**:检查网络;等一会儿再试;还不行就重启 Trae 重登。
- **报错提示额度/账号问题**:举手找助教,可能是登录或额度配置问题。

> 💡 记住 SOLO 的一个关键能力:**它能自己新建文件、往里写内容、保存到你的工作区。** 所以后面写 PRD,你不用手抄进文件——一句话让它建、让它存就行(实操 4 会用到)。
>
> 提示:下一步(实操 2)先用 SOLO 做个纯前端小页面热身、尝个甜头;之后(实操 3)再拿它来梳理 C/S 图书馆的真需求。

实操 2 · 城市探索热身(纯前端上手)

上一步你确认了 SOLO 能用,先别急着上 C/S 图书馆——**这是你第一次用 Trae 的上手热身:今天先用 SOLO 做个能看、能点的纯前端小页面(城市探索指南),尝个甜头、建立信心。** 这个热身是**纯前端网页**(HTML/CSS/JS,不碰后端、不用框架、不接数据库),几分钟出活,只为让你第一次感受"你说人话 → AI 做出真页面"。**这个热身只今天做这一次,后面就直接上我们的桌面 C/S 图书馆(Swing + Socket + JDBC)了。**

先看一眼成品:打开同目录《城市探索-热身案例-答案版.html》,心里有个目标样子——渐变首屏 + 搜索框、一排分类卡片、几张推荐地点卡片、点卡片弹详情。下面用 SOLO 分四步把它做出来。

分步原则(今天开始每一步都用,后面搭 C/S 图书馆也一样):**一步一个 prompt、说具体(颜色/数量)、"只做 XX 不动其他"。**

步骤 2.1 · 起项目 + 渐变首屏 + 搜索框

【操作】

- 在 SOLO 对话框里,**原样复制**下面这段话输入并回车:

```
帮我做一个"城市探索指南"的纯前端小网页(只用 HTML+CSS+JS,不要后端、不要框架)。
先做整体:一个渐变紫色的首屏,标题"城市探索指南",下面一个圆角搜索框。
```

【✅ 怎么算做好】

- 预览里能看到一个带**渐变紫色首屏**的页面,上面有标题"城市探索指南"和一个**圆角搜索框**。

【卡住怎么办】

- **没有预览 / 看不到效果**:让 Trae 把生成的 HTML 保存成文件后再预览;或直接在浏览器里打开它生成的 `.html` 文件看效果。
- **样子不对(比如没渐变、没搜索框)**:用一句话描述差在哪让它改,比如"首屏背景改成紫色渐变,标题下面加一个圆角搜索框,别的不动"。

步骤 2.2 · 分类卡片

【操作】

- 承接上一步,在同一会话里**原样复制**输入并回车:

```
基于现有页面,在首屏下面加一排分类卡片:美食🍜、风景🏞️、咖啡☕、文艺🎨、购物🛍️,
每张显示 emoji + 分类名 + 数量,鼠标悬停有上浮效果。只加分类,别动首屏。
```

【✅ 怎么算做好】

- 首屏下面出现一排**分类卡片**(美食/风景/咖啡/文艺/购物),每张有 emoji、分类名和数量;鼠标移上去卡片**轻微上浮**。首屏没有被改动。

【卡住怎么办】

- **它把首屏也改了**:回一句"你把首屏也动了,请恢复首屏原样,只保留新加的分类卡片"。
- **没有悬停上浮效果**:回一句"给分类卡片加鼠标悬停上浮效果(hover 时轻微向上移动),别的不动"。

步骤 2.3 · 推荐卡片列表

【操作】

- 承接上一步,在同一会话里**原样复制**输入并回车:

```
基于现有页面,在分类下面加"本周热门推荐",卡片网格展示 6 个地点:
每张有渐变色封面(色块代替图片)+ 地点名 + 分类标签 + 一句描述 + 评分。
数据你先编 6 条合理的(如"巷子口牛肉面/美食/4.8/开了20年的老店")。只加推荐区。
```

【✅ 怎么算做好】

- 分类下面出现"本周热门推荐",一个**卡片网格**里有 **6 个地点**,每张卡片有渐变色封面、地点名、分类标签、一句描述和评分。前面的首屏和分类没被改动。

【卡住怎么办】

- **只出来几条 / 数量不对**:回一句"推荐卡片要正好 6 张,数据你补齐到 6 条,别的不动"。

- **卡片没有封面色块**:回一句"每张推荐卡上方加一块渐变色封面(用色块代替图片),别的不动"。

步骤 2.4 · 详情弹窗 + 搜索过滤

【操作】

- 承接上一步,在同一会话里**原样复制**输入并回车:

```
基于现有页面,加两个交互:① 点推荐卡片弹详情弹窗(封面/名字/标签/描述 + 关闭按钮);
② 搜索框输入关键词回车,按地点名或分类过滤推荐列表。只加这两个交互,别改布局。
```

【✅ 怎么算做好(实操 2 的验收点)】

- **首屏 + 分类 + 推荐卡都在**;
- **点一张推荐卡片,弹出详情弹窗**(有封面、名字、标签、描述和关闭按钮),点关闭能收起;
- **搜索框里输入关键词回车,推荐列表能按地点名或分类过滤。**
- 对照《城市探索-热身案例-答案版.html》的样子,大体一致——热身就算成了。

【卡住怎么办】

- **点卡片没反应 / 弹窗关不掉**:用**纠错三步法**(下面理论环节会讲)描述清楚让它改,比如"点推荐卡片应该弹详情弹窗,现在点了没反应,请修好这一处,别改布局"。
- **搜索没过滤效果**:回一句"搜索框输入关键词回车后,推荐列表要按地点名或分类过滤,现在没过滤,请修好这一处"。
- **样子和答案版差很多**:不用一模一样,只要首屏、分类、推荐卡、点卡弹详情、搜索能过滤这几样都在,就达标了。

🔬 承上启下(这一步的意义)

感受到了吧:你随口说,AI 就做出来了——这就是让 AI 写代码的直观体验。但你可能也注意到:让它"编 6 条合理数据",它编的和你想的不一定一样;你没说清颜色,它自己配的。热身是网页、几步就成;可我们今天真正要做的是**能局域网多机跑、要防并发超借的桌面 C/S 图书馆**,复杂得多。**AI 能做,但要做得又准又全、做成能交付的真系统,光"随口说"不够——得给它一份清清楚楚的"图纸"。** 这份图纸怎么写,就是接下来 C/S 图书馆要用到的东西:先理清需求,再让 AI 把它整理成 **Agent PRD**。下面几个实操就来做这件事。

实操 3 · 反向提问梳理 C/S 图书馆需求

新手最大的问题是"想不全"——只想到"做个图书馆系统",一堆细节没考虑到,尤其是 C/S 特有的东西(架构分几段?客户端和服务端怎么通信?多人同时用怎么办?)。有个好办法:**让 AI 反过来问你**,把需求一个一个问出来。

步骤 3.1 · 在 SOLO 里输入反向提问的话

【操作】

- 在 SOLO 对话框里,**原样复制**下面这段话输入并回车:

```
我要做一个校园图书馆借阅系统,是客户端-服务器的桌面程序,但需求没完全想清楚。
请你像产品经理一样,每次只问我一个问题,一共问 6-8 个,帮我把需求理清楚;我回答完你再问下一个。
```

- AI 会开始**一次只问你一个问题**,你回答完它再问下一个。

【✅ 怎么算做好】

- AI 没有一上来就写代码,而是**开始一个一个地问你问题**。

【卡住怎么办】

- **AI 一次问了一大堆 / 直接开始写代码**:回它一句"请一次只问我一个问题,我答完你再问下一个",把它拉回节奏。
- **AI 问的问题跑题**:正常继续答,或提醒它"我们现在只梳理这个 C/S 图书馆借阅系统的需求"。

步骤 3.2 · 一问一答,把需求理清楚

【操作】

- AI 问一个,你答一个。下面这张**对照表**告诉你:它大概会问什么、你可以怎么答。照着自己的想法回答即可(答案不必和表里一字不差,但方向对上)。

AI 大概会问什么	**你可以怎么答(这些答案就是需求)**
系统给谁用?有哪几种角色?	**管理员** + **读者** 两种角色
图书要记录哪些信息?	ISBN / 书名 / 作者 / 出版社 / 分类 / 总藏数 / 可借库存(stock)
借阅要记录哪些信息?	借书人 / 所借图书 / 借出时间 / 应还时间 / 归还时间 / 状态(borrowed/returned)
借书有什么规则?	先校验(书在、可借库存 > 0、读者没借满)→ 可借库存减 1 → 写一条借阅记录;每人借阅上限默认 5 本;应还日期 = 借出当天 + 30 天
用什么技术、什么架构?	**Java 11 + Swing 客户端 + Socket + 多线程服务端 + JDBC + MySQL;档 B 三段式,客户端不直连数据库、一律经 Socket 请求服务端**
客户端和服务端怎么通信?	自定义**文本协议**,一行一条 `命令|参数|参数`;成功回 `OK|数据`,失败回 `ERR|原因`(如 `LOGIN|用户名|密码`)
多个读者同时在线怎么办?	服务端 `accept()` 到一个客户端就**开一个线程**(每客户端一线程);借书并发用事务 + `WHERE stock>0` 防超借
界面要什么风格?几个窗口?	登录窗(**含服务器地址 + 端口输入**,为局域网)、读者主窗、图书详情、管理员主窗、监控窗口……**≥10 个窗体**;主色蓝、用 JTable 展示列表

【✅ 怎么算做好(实操 3 的验收点)】

- **本来只有"做个图书馆系统"一句话,经 AI 一问一答之后,需求被理清楚了**:角色、图书/借阅字段、借书规则、**技术架构(档 B 三段式)**、**通信协议**、**并发(每客户端一线程)**、界面窗口——都有答案了。这一问一答的记录留在对话里就行,下一步让 Trae 直接根据它整理成 PRD。

【卡住怎么办】

- **有的问题不知道怎么答**:照上面对照表里的答案答就行,这是本项目约定好的标准需求。
- **AI 问的顺序和表里不一样**:没关系,顺序不重要,只要这几类信息都答到了就行。
- **AI 没问到"架构/通信协议/并发"这几条**:主动补给它——"补充一下:这是客户端-服务器程序,客户端不直连数据库,通过 Socket 文本协议连多线程服务端,服务端每个客户端开一个线程。" 这三条是 C/S 的命根子,一定要进需求。

实操 4 · ⭐ 让 Trae 生成并保存 Agent PRD

需求问清楚了,**不用你手动整理、手动粘贴**——直接让 Trae 干:根据刚才的问答,把需求整理成一份结构完整的 **Agent PRD**,并**自己建文件保存**。(注意:这里存的是**真项目 C/S 图书馆**的图纸,和实操 2 那个纯前端热身页面无关。)

Agent PRD 是什么:不是写给人看的文档,是**写给 AI 看的项目说明书**。它把项目的一切规范固定下来——目标、技术选型/架构、数据模型、**通信协议**、业务规则、**并发多线程**、编码规范、界面清单。放进 Trae 的 Rules 文件(`.trae/rules`)后,Trae **每次生成代码都会自动读它、照它来**。

步骤 4.1 · 在 SOLO 里让 Trae 整理并保存

【操作】

- 承接实操 3 的对话,在同一个 SOLO 会话里,**原样复制**下面这段话输入并回车:

```
把我们刚理清的需求,整理成一份项目规范,保存到项目根目录的 .trae/rules 文件。要包含:
① 项目概述 ② 技术选型(Java Swing + Socket + JDBC + MySQL,档B三段式C/S,客户端不直连库;
服务端端口 9000)
③ 数据模型(t_user / t_category / t_book / t_borrow 四张表 + 字段 + 主外键)
④ 通信协议(一行一条 命令|参数,成功 OK|数据、失败 ERR|原因;含
LOGIN/BOOK_QUERY/BORROW/RETURN 等)
⑤ 业务规则(角色权限;借书要在同一事务里减库存+写记录、防超借用 WHERE stock>0 判影响行数、借阅上限)
⑥ 并发与多线程(服务端每客户端一线程)⑦ 编码规范(分层、PreparedStatement 防注入、UTF-8)
⑧ 界面清单(≥10 个窗体,登录窗含服务器地址+端口)。
```

- Trae 会**自己整理这八块内容,并新建 `.trae/rules` 文件、把内容写进去、保存**。

【✅ 怎么算做好(实操 4 的验收点)】

- **Trae 生成了内容,并且在你的工作区里出现了 `.trae/rules` 文件**(左边文件区能看到 `.trae` 文件夹,里面有 `rules` 文件),打开它能看到刚整理的八块 PRD。到这一步,PRD **是 Trae 根据你的问答自己整理、自己存的,你一个字没抄。**

【卡住怎么办】

- **Trae 只把内容打在对话里,没建文件**:回它一句"请把上面这份规范保存成项目根目录下的 `.trae/rules` 文件,不用只贴在对话里"。

- **建不了以点开头的文件夹**:有的系统会把 `.trae` 当隐藏文件夹,这是正常的;在 Trae 文件区里通常能正常看到。若确实没出现,回一句"请确认已经在工作区根目录创建了 `.trae/rules` 并写入内容"。
- **不确定 Rules 放哪**:本课统一放项目里的 `.trae/rules`;如果你的 Trae 版本有专门的 Rules 设置入口,让它放那个入口也可以,内容一样。

实操 5 · 检查 + 微调(审查是你的活)⭐

Trae 整理得快,但**不一定全对**——尤其 C/S 项目有几条 AI 特别容易漏。这时候你的判断力就派上用场了:你知道 C/S 图书馆的命根子是"通信协议 + 每客户端一线程 + 借书事务",AI 漏了你要能看出来。**让 AI 干活、你把关,这就是"信任但验证"。**

步骤 5.1 · 打开 `.trae/rules`,对照标准审查

【操作】

- 打开 Trae 刚生成的 `.trae/rules`,和本节末尾附的 **PRD 标准版**(抓手1,本项目约定好的标准图纸)逐块对照,重点看**三件 AI 最容易漏的事**:
  1. **架构对不对?** 是不是写明了**档 B 三段式、客户端不直连数据库、一律经 Socket 请求服务端**?
     - AI 常偷懒把客户端直接写成 JDBC 连库(那就退回了档 A 两层)。技术选型里必须写死:Swing 客户端 ⇄ Socket 文本协议 ⇄ 多线程服务端 ⇄ JDBC ⇄ MySQL,端口 9000。
  2. **通信协议在不在?** 有没有明确"一行一条 `命令|参数`、成功 `OK|数据` / 失败 `ERR|原因`",并列出主要命令(`LOGIN`、`BOOK_QUERY`、`BORROW`、`RETURN`、`MY_BORROW` 等)?
     - 协议是 C/S 的命根子,漏了后面所有联调都乱。
  3. **这三条业务/并发规则写死了没有?**(AI 最容易漏)
     - **借书事务**:减库存(`stock-1`)和写借阅记录必须在**同一个事务**里(要么全成、要么全滚)。
     - **防超借**:并发下用 `UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0`,按**影响行数**判定是否借到最后一本——影响行数为 0 就说明没库存了。
     - **每客户端一线程**:服务端 `accept()` 后为每个客户端开一个线程(`new Thread(new ClientHandler(socket)).start()`),多读者同时在线互不阻塞。

【✅ 怎么算做好】

- 你**逐块对照**过了,心里清楚 Trae 这份 PRD **哪些对、哪些缺、哪些被改错了**。

【卡住怎么办】

- **不知道对照什么**:就照本节末尾的"PRD 标准版"一块一块比,标准版有的、你这份没有,就是缺的。
- **看不懂某一块在说什么**:让 Trae 用大白话解释——"请用大白话解释一下这份 rules 里第五块业务规则的借书事务是什么意思,为什么减库存和写记录要在一个事务里"。

步骤 5.2 · 缺的、错的,用纠错三步法让 Trae 补

【操作】

- 发现缺什么、错什么,**别自己动手改文件**,用**纠错三步法**让 Trae 补:①定位(哪块不对)→ ②描述(应该是什么、现在是什么)→ ③让它只改那一处。
- 示例一(假设发现借书规则没写事务):

```
.trae/rules 的业务规则里,借书没写"必须在同一个事务里"。
请补一条:借书时"可借库存 stock -1"和"写借阅记录"必须在同一个事务(要么全成、要么全滚),
并发防超借用 UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0 判影响行数。
只改这一处,别的不动。
```

- 示例二(假设发现并发/多线程那节漏了):

```
.trae/rules 的并发规则请补一条:服务端 ServerSocket.accept() 到一个客户端后,
要 new 一个线程(ClientHandler)去处理它,做到"每客户端一线程",多个读者同时在线互不阻塞。
只加这一条,别的不动。
```

- 示例三(假设发现架构被写成客户端直连库):

```
.trae/rules 的技术选型写成了客户端直接用 JDBC 连数据库,这不对。
请改成档 B 三段式:客户端只用 Socket 发文本请求给服务端,由服务端用 JDBC 连 MySQL,
客户端不直连数据库。只改技术选型这一块。
```

【✅ 怎么算做好(实操 5 的验收点)】

- **`.trae/rules` 里,架构是档 B、通信协议在、借书事务/防超借/每客户端一线程这三条关键规则都在,并已保存。** 到这一步,你手里就有了一份**经过你审查、把关过的项目图纸**——后面每次让 AI 生成代码,它都会自动读这份规则照着来。

【卡住怎么办】

- **改完又冒出别的地方乱了**:每次只让 AI 改一处、说清"别的不要动";乱了就回一句"你刚才把 XX 也改了,请把 XX 恢复,只保留这一处改动"。
- **不确定改对没有**:改完再打开 `.trae/rules` 对着标准版看一眼那一块,确认补上了。

> 🔬 **信任但验证**:AI 整理得快,但**对不对要你判断**。C/S 项目的三条命根子(协议 / 每客户端一线程 / 借书事务),就是你今天审查的重点。你手里有一双"判断 AI 的眼睛",这是把课上到能审查 AI 的关键。

附 · PRD 标准版(审查参照,不用抄)

> 下面这份是本项目约定好的 **C/S 图书馆 Agent PRD 标准版**(抓手1),只作**审查对照**用——拿它比 Trae 生成的那份哪里缺、哪里被改错。**不需要你手抄进文件**(文件是实操 4 让 Trae 生成的)。它对应的成品 = 答案版 `05-projects/cs-swing-library-demo/`。

```
# C/S 图书馆借阅系统 — 项目规范(Agent PRD / Rules)

## 一、项目概述
校园图书馆借阅系统,客户端-服务器(C/S)桌面程序,分管理员和读者两种角色:
- 管理员:管理图书、分类、读者;查看借阅记录和超期统计。
- 读者:登录后查询图书、借书、还书、看自己的借阅记录。
MVP:图书 CRUD + 分类 + 读者 + 登录 + 借还书(事务)+ 搜索 + 统计,本地跑通、局域网多客户端可演示。

## 二、技术选型(严格遵守,不许自作主张换)
```

- 语言/界面:Java 11 + Swing;禁 JavaFX/Web/前端框架。
- 架构:档 B 三段式 —— Swing 客户端 ⇄ TCP Socket 文本协议 ⇄ 多线程服务端(每客户端一线程)⇄ JDBC ⇄ MySQL。客户端不直连库。
- 数据库:MySQL 8,库名 library_cs,连接 127.0.0.1:3306 / root / 12345678;JDBC url 用 characterEncoding=UTF-8(不是 utf8mb4,那是字符集名会报错)。
- 依赖:除 MySQL JDBC 驱动外不引第三方库;JSON/编码用手写极简工具类。服务端端口 9000。

三、数据模型(五张核心表)
- t_user:id, username(唯一), password, real_name, role(admin/reader), max_borrow(默认5)
- t_category:id, name
- t_book:id, isbn, title, author, publisher, category_id, total, stock(可借库存)
- t_borrow:id, user_id, book_id, borrow_time, due_time, return_time, status(borrowed/returned)
- t_fine(可选):id, borrow_id, amount, paid

外键:t_book.category_id→t_category.id;t_borrow.user_id→t_user.id;t_borrow.book_id→t_book.id。

种子:3 分类、6-8 图书、admin/123456、zhangsan/123456、lisi/123456。

四、通信协议(C/S 核心,一行一条 命令|参数)
- 响应:成功 OK|数据;失败 ERR|原因。数据含 | / 换行要转义。
- 命令:LOGIN|用户名|密码、REGISTER|..、BOOK_QUERY|关键字|分类id、CATEGORY_LIST、BORROW|userId|bookId、RETURN|borrowId、RENEW|borrowId、MY_BORROW|userId、BOOK_ADD|json、BOOK_UPDATE|json、BOOK_DELETE|id、READER_LIST、BORROW_RECORD。
- 登录响应:OK|role|realName|userId / ERR|账号或密码错误。

五、业务规则(生成代码必须遵守)
1. 角色权限:只有 admin 能增删改图书/分类、管理读者、看全部借阅统计;reader 只能查询/借还自己的。
2. 借书:校验(书在、可借库存>0、未还数<max_borrow)→ 库存 -1 → 写记录(due=今+30天)。
   减库存和写记录必须在同一事务。防超借:UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0 按影响行数。
3. 还书:防重复归还 → 标 returned + 库存 +1,同样用事务。
4. 登录:用户名密码校验(PreparedStatement)+ 角色分流(admin/reader 打开不同主窗)。
5. 安全底线:密码本课可先明文,但代码留注释标明生产必须 BCrypt;SQL 一律 PreparedStatement 防注入;服务端独立校验。

六、并发与多线程(C/S 关键,答辩会问)
- 服务端 ServerSocket.accept() 后每个客户端开一个线程 ClientHandler,多读者同时在线互不阻塞。
- 客户端耗时请求放子线程(SwingWorker),不卡界面(EDT)。
- 借书并发一致性靠事务 + WHERE stock>0 影响行数。

七、编码规范
- 分层:client/ui(界面)+ client/net(Socket 收发);server(监听/Handler)+ server/service(业务/事务)+ server/dao(JDBC)+ server/util(DBUtil);common/entity + common(协议与编码工具)。
- SQL 不写在界面事件里;关键代码写中文注释(事务、并发、协议)。UTF-8 统一;异常 try-catch + JOptionPane 友好提示。

八、界面清单(≥10 个窗体,大纲硬指标)

```
- 登录窗(含服务器地址+端口,支持局域网)、读者主窗(图书查询/我的借阅)、图书详情+借书、
  管理员主窗(图书管理/分类/读者/借阅记录/统计)、关于窗;主色蓝,JTable 列表,布局用布局管理器不用绝对定位。
- (加分)服务端监控窗口:监听端口/在线客户端数/实时日志/启停按钮。

## 九、当前任务(每次只给一个,分步来)
(空——后面每一步 SOLO prompt 里写"当前要做什么",见分步脚本)
```

八块速记小抄(帮你审查时定位):

块	管什么	一句话
一、项目概述	做什么、给谁用	校园图书馆 C/S,管理员 + 读者
二、技术选型	用什么技术、什么架构	Java Swing + Socket + JDBC + MySQL,**档 B 三段式**
三、数据模型	有哪些表、哪些字段	t_user / t_category / t_book / t_borrow 四张(+t_fine 可选)
四、通信协议	客户端服务端怎么说话	一行 `命令|参数`,`OK|数据` / `ERR|原因`
五、业务规则	借还书、权限怎么算	借书减库存+写记录**同一事务**、防超借、借阅上限
六、并发多线程	多人同时用怎么办	**每客户端一线程** + 事务防超借
七、编码规范	代码怎么组织	分层、PreparedStatement 防注入、UTF-8
八、界面清单	有哪些窗口	≥10 窗体,登录窗含服务器地址

> 📌 **顺带看一眼答案版长什么样**(帮你理解这份 PRD 会"施工"成什么):后面单元②建库时,`t_book` 的 `stock` 就是可借库存;单元③借书那段,答案版 `server/service/BorrowService.java` 里正是 `conn.setAutoCommit(false)` 开事务、`UPDATE t_book SET stock = stock - 1 WHERE id = ? AND stock > 0` 按影响行数防超借,三步全成才 `commit()`。你今天在 PRD 里写下的规则,单元③就要用它来检查 AI 写得对不对。

🔬 拓展理解:为什么 PRD 比代码更重要

打个比方:**规范(PRD)= 图纸,代码 = 施工。**

- 你要做的是**画好、审好图纸**;施工(写代码)交给 AI。
- 图纸好,AI 可以**反复施工**——重新生成、改了需求再生成,都不走样;图纸含糊,AI 再厉害也做不出你要的东西。
- 尤其 C/S 项目,架构(几段式)、通信协议、并发这些**一旦在图纸里定死**,AI 就不会把客户端写成直连库、不会漏掉多线程——**所以在 AI 时代,你写好、审好的这份需求文档,就是新时代的源代码**。这就是"**规范是新的源代码**"。

今天你没写一行业务代码:需求是你理清的、整理落盘是 Trae 干的、审查把关是你做的。从下个单元起,就拿这份图纸,让 AI 一块一块把 C/S 图书馆"施工"出来。你会发现:**规范写得越清楚、审得越准,AI 生成得越准。**

今天做完了什么(自检清单)

- [ ] Trae SOLO 能用:能输入、AI 有回应,并且它能自己建文件/存盘(实操 1)
- [ ] 城市探索热身页面做出来了:首屏 + 分类 + 推荐卡都在,点卡片弹详情、搜索能过滤(实操 2)
- [ ] 用反向提问法把 C/S 图书馆需求理清楚了:角色/字段/借阅规则/**架构(档 B)/通信协议/并发(每客户端一线程)**/界面都有答案(实操 3)
- [ ] 让 Trae 生成并保存了 `.trae/rules`,里面是它整理的八块 PRD(实操 4)
- [ ] 对照标准版审查过:架构是档 B、通信协议在、借书事务/防超借/每客户端一线程三条都在(实操 5)

五项都打勾,今天的目标就达成了——**给 AI 的图纸,由你理清需求、由 Trae 落盘、由你审查把关,备好了。**

课后任务

用今天学的两招——**反向提问法** + **让 Trae 生成并保存 Rules**,为"**你自己想做的一个小型 C/S 系统**"写一份 Agent PRD。

怎么做:

1. 想一个你感兴趣的、能做成客户端-服务器桌面程序的小系统(比如:局域网聊天室、简单的多人抢票、宿舍设备预约、二手书交易……随你)。先想一想:它该走**档 A 两层**还是**档 B 三段式**?(要多人同时联网、要并发一致 → 档 B;单机小工具 → 档 A。)
2. 在 Trae SOLO 里输入反向提问的话(把"校园图书馆借阅系统"换成你的系统):

   ```
   我要做一个 xxx 系统,是客户端–服务器的桌面程序,但需求没完全想清楚。请你像产品经理一样,
   每次只问我一个问题,一共问 6–8 个,帮我把需求理清楚;我回答完你再问下一个。
   ```

3. 一问一答,把需求理清楚(记得答清楚:几段式架构、通信协议、多人并发怎么办)。
4. 让 Trae 把需求整理成一份八块结构(概述 / 技术选型 / 数据模型 / 通信协议 / 业务规则 / 并发多线程 / 编码规范 / 界面清单)的 Agent PRD,**并保存到那个项目的 `.trae/rules` 文件里**(参照实操 4 的 prompt,把内容换成你的系统)。
5. 打开生成的 `.trae/rules`,**自己审查一遍**——架构档次对不对、通信协议在不在、并发有没有考虑,缺的用纠错三步法让 Trae 补(参照实操 5)。

交付:一份你自己系统的 Agent PRD(八块齐全、你审查过),存在该项目 `.trae/rules` 里,下次课带来。

> 记住今天最重要的一句话:**规范是新的源代码。** 想清楚、说清楚、审清楚图纸,比催 AI"再好一点"有用得多。